

Kernel Enhancement Audit & Hardening Plan (ghatana commit 4431520ab590)

1. Executive Summary

This document reviews the Kernel subsystem in the `ghatana` monorepo, focusing on the integration between the Digital-Marketing product (`DMOS`) and the underlying kernel modules, extensions, and plugins. It also assesses the Kernel boundary hardening plan and plugin architecture guidelines provided in `KERNEL_MODULE_EXTENSION_PLUGIN_DECISION_GUIDE.md` and `KERNEL_BOUNDARY_HARDENING_PLAN.md`. The goal is to ensure that DMOS leverages kernel capabilities appropriately, that kernel APIs remain stable, and that product-specific logic does not leak into the platform.

Findings: The kernel provides a flexible plugin architecture with **KernelModule** for domain ownership, **KernelExtension** to enrich existing modules, **KernelPlugin** for hot-swap functionality, and **KernelLifecycleAware** for service lifecycles ¹. Platform plugins such as audit trail, ledger, consent, compliance, fraud, and risk are available ². However, the current DMOS implementation uses these plugins inconsistently: duplicate boundary resolver classes exist; risk and compliance plugins are product-specific rather than generic; core observability is not wired; and some plugin APIs are duplicated across modules ³. The kernel hardening plan outlines phased tasks to consolidate boundaries, unify plugin APIs, generalise risk/compliance engines, and wire observability ⁴. These tasks are only partly implemented.

2. Kernel Inventory

2.1 Kernel Modules and Ports

Module	Purpose	Ports/Interfaces	Implementation	DMOS Usage
Audit Module	Captures audit events for domain actions	<code>AuditTrailPort</code> for recording events; <code>AuditTrailPlugin</code> for plugin implementation	Provided by core platform; plugin logs events and persists to audit table	DMOS uses <code>recordAudit</code> via <code>DmKernelBridge</code> log approvals, campaigns, strateg

Module	Purpose	Ports/Interfaces	Implementation	DMOS Usage
Authorization Module	Determines whether actions are allowed based on roles, permissions, policies	<code>AuthorizationPort</code> ; plugin implements RBAC/ABAC	Kernel includes default RBAC plugin; product can override via policy pack	DMOS defines <code>DmPolicyPack</code> and uses kernel bridge to authorize campaign strategies
Consent Module	Manages user consent lifecycle	<code>ConsentPort</code> ; plugin stores consent proof; ensures data usage is compliant	Platform plugin available but not fully integrated into DMOS	DMOS references consent in compliance rules but does not invoke <code>ConsentPort</code>
Compliance Module	Runs rule packs for compliance (e.g., marketing preflight checks, brand guidelines)	<code>CompliancePort</code> ; policies loaded from <code>DomainPack</code> rule sets	DMOS includes <code>DmComplianceRules</code> ; uses <code>DmKernelBridge.runCompliance</code>	Product defines its own risk/compliance plugin; duplicates logic present in DMOS
Risk Module	Executes risk scoring and fraud detection	<code>RiskPort</code> ; plugin returns risk levels for actions	DMOS includes <code>DmRiskPlugin</code> but marks it as placeholder; always returns risk level <code>LOW</code>	DMOS uses <code>DmRiskPlugin</code> for campaign approval
Ledger/Accounting Module	Handles financial transactions and ledger entries	<code>LedgerPort</code> ; plugin posts entries to ledger store	DMOS may need to track marketing spend and ROI	Not yet used by DMOS
Approval Module	Coordinates human approval workflows	<code>HumanApprovalPort</code> ; plugin persists approvals and notifies approvers	DMOS uses <code>HumanApprovalPlugin</code> to manage approval requests	Implementation partially generic; but product uses local <code>DmApprovalSnapshot</code> structure
Observability Module	Provides metrics, logs, and tracing ports	<code>MetricCollectorPort</code> , <code>TracingPort</code> , <code>LoggingPort</code>	Kernel has baseline but not yet enforced; KIP emphasises wiring OTel before telemetry ⁵	DMOS logs events and uses a local metric collector that writes logs

2.2 Kernel Plugins & Lifecycle

DMOS extends the kernel via a **KernelBridge** implemented in `dm-kernel-bridge`. It provides typed adapters for authorization, compliance, risk, approval, audit, consent, ledger, metrics, and health. The bridge uses platform plugin providers to obtain ports and delegates calls. DMOS also defines a **DomainPack** (`DmDomainPack`) with rule sets (`DmComplianceRules`, `DmConsentRules`, `DmRiskRules`) and registers them via `domain-pack.xml`.

The kernel hardening plan identifies the following plugin boundary issues:

- **Duplicate boundary resolvers** – multiple classes resolve domain boundaries (e.g., `DmosBoundaryResolver`, `GenericBoundaryResolver`), causing duplication and drift ³.
- **Plugin API duplication** – product uses bespoke plugin interfaces (e.g., `DmApprovalPlugin`) instead of generic kernel ports ⁴.
- **Product-specific risk/compliance** – risk and compliance logic exists in DMOS only; should be generic across products ⁶.
- **Observability unwired** – `MetricCollectorPort` not wired; `TelemetryPort` missing; no context propagation ⁵.
- **Policy resolution** – boundary resolvers embed region, product, and module into path; should use rule-based policy model ⁷.

3. Kernel Hardening Progress

The hardening plan outlines a phased approach:

Phase	Description	Status in DMOS
Phase 0 – Build & API baseline	Fix composite build paths; ensure plugin classes unique; remove duplicate resolver classes; run test quality gates; unify plugin API naming	Some improvements: plugin names are consistent; DMOS uses central kernel bridge; build passes. However, duplicate boundary resolvers still exist (e.g., <code>DmPolicyResolver</code> vs. <code>GenericPolicyResolver</code>), and quality gate is violated by <code>UnsupportedOperationException</code> . Test purity scanning not yet integrated.
Phase 1 – Boundary consolidation & canonical APIs	Consolidate boundary resolvers; align plugin APIs; create rule-based policy model; unify error codes; generalise risk plugin	Not started. DMOS still defines its own policy resolver and risk plugin. Rule model is domain-specific; risk plugin returns constant risk.

Phase	Description	Status in DMOS
Phase 2 – Plugin purity & platform generalisation	Refactor risk, compliance, approval, ledger, consent into generic engines; remove product-specific plugin logic; rename ledger plugin; add encryption, retention, DSAR support	Partially started. DMOS uses <code>HumanApprovalPlugin</code> but snapshot model is product specific. Risk and compliance remain product specific. Ledger integration missing.
Phase 3 – Product pack validation & runtime bridging	Validate domain packs at build time; ensure rule pack integrity; run startup wiring integration tests; add kernel-owned ports to allocate context and correlation IDs; update CLI/test harness	Not started. Domain pack validation is not automated; DMOS does not test plugin startup wiring; correlation IDs not globally propagated.
Phase 4 – Observability & security	Wire OpenTelemetry ports before enforcement; ensure context propagation; add Kernel-owned ports for encryption, hashing, redaction, DSAR; enforce plugin purity scanning	Partially started. Observability baseline logs metrics, but OTel not integrated; encryption/hashing ports not used.
Phase 5 – Documentation & matrices	Document module/extension/plugin guidelines; publish boundary policy mapping; maintain traceability matrices	Not started. Only the decision guide and hardening plan exist.
Phase 6 – Strategic backlog	Add configuration service, event registry, stable rule DSL, generic CLI, plugin marketplace	Not started.

4. Kernel & DMOS Integration Gaps

- **Policy resolver duplication** – DMOS defines its own `DmPolicyResolver` and boundary lookup logic; this duplicates kernel logic and may diverge from platform standards. Consolidate into a single `PolicyResolver` that uses a rule-based DSL as planned ⁷.

- **Risk/compliance engines** – DMOS risk and compliance plugins are simplistic and product specific. Kernel should provide generic scoring and rule execution engines; DMOS should supply rule sets without hardcoding risk levels.
- **Observability ports** – DMOS logs metrics via `DmosMetricsCollector` but does not implement `MetricCollectorPort`. Kernel should provide `MetricCollectorPort` and `TracingPort`, and DMOS should implement them or reuse the platform implementation.
- **Encryption & hashing** – Kernel lacks a generic `EncryptionPort` / `HashingPort`; DMOS stores raw PII. The hardening plan suggests adding ports for encryption, hashing, redaction, DSAR; these are still missing.
- **Rule pack validation** – Domain packs can include invalid or duplicate rules; there is no build-time or startup validation. Add validation step to detect duplicate keys, missing handlers, or invalid policies.
- **Context propagation** – Correlation and tenant context are not propagated across kernel and DMOS services; logs/traces cannot link front-end requests to backend events. Add context propagation support to kernel ports.
- **Plugin hot-swap** – Kernel plugins are intended to be hot-swappable; DMOS does not implement plugin lifecycle management; update to use `KernelPlugin` lifecycle methods (`install`, `uninstall`, `reload`).
- **Ledger integration** – DMOS does not use the ledger plugin; budgets and ROI calculations are not recorded. Integrate with ledger to track financial transactions.

5. Recommended Kernel Enhancement Tasks

Priority	Area	Task	Evidence	Acceptance Criteria	Required Tests
High	Duplicate resolvers	Consolidate <code>DmPolicyResolver</code> and generic policy resolvers into a single canonical class; remove duplication from DMOS; ensure kernel supplies rule-based DSL for policy resolution ⁷	Resolvers are unified; DMOS imports and uses kernel resolver; no duplicate classes remain	Unit test ensures rules map correctly to policies; integration test verifies policies executed via kernel without DMOS override	
High	Generic risk engine	Refactor <code>DmRiskPlugin</code> into generic <code>RiskPlugin</code> that uses rule sets defined in domain packs; return dynamic risk scores; remove product-specific logic ⁶	DMOS supplies rule set; kernel computes risk; risk scores vary based on attributes; DMOS does not code risk logic	Unit test for risk engine returning correct scores; integration test verifying risk gating in DMOS functions	

Priority	Area	Task	Evidence	Acceptance Criteria	Required Tests
High	Generic compliance engine	Move compliance logic into kernel; DMOS contributes <code>DmComplianceRules</code> as policy definitions; unify error codes and severity classifications	<code>CompliancePort</code> returns rule failures with codes/severity; DMOS handles errors uniformly	Tests verifying rule failure detection; integration test verifying preflight blocking	
High	Observability ports	Implement <code>MetricCollectorPort</code> and <code>TracingPort</code> ; update <code>DmosMetricsCollector</code> to delegate to kernel; propagate correlation IDs across calls ⁵	Metrics exported via kernel; OTel traces captured across modules; correlation IDs appear in logs	Instrumentation test capturing metrics; correlation propagation test	
High	Encryption & hashing ports	Add <code>EncryptionPort</code> and <code>HashingPort</code> to kernel; implement default AES and HMAC hashing; update DMOS to use them for PII storage	PII stored hashed/encrypted; DSAR operations supported; secrets not logged	DB integration test verifying hashed values; DSAR test	
Medium	Rule pack validation	Add build-time and startup validation for domain packs: check duplicate rule keys, missing handlers, invalid DSL syntax; fail build or log errors	Validation runs in CI; invalid packs cause build failure; DMOS passes validation	Unit tests for validation; integration test with invalid pack	
Medium	Plugin lifecycle management	Implement <code>KernelPlugin</code> lifecycle methods (<code>install</code> , <code>uninstall</code> , <code>reload</code>) for DMOS plugins; enable hot-swap; ensure state is preserved across reload	Plugins can be reloaded without losing context; metrics and audit remain intact	Tests invoking reload and verifying plugin functionality persists	
Medium	Ledger integration	Expose <code>LedgerPort</code> to DMOS; implement <code>DmLedgerPlugin</code> to record marketing spend; track ROI; connect to accounting system	Budgets are recorded; spend vs. ROI reported; ledger entries created for each spend	Integration test verifying ledger entries created; ROI calculations	

Priority	Area	Task	Evidence	Acceptance Criteria	Required Tests
Medium	Policy DSL & configuration service	Provide a DSL for writing policies (risk, compliance, consent); central configuration service to load policies at runtime; update DMOS to consume policies via configuration	Policies can be updated without code changes; DMOS fetches policies via service; no duplication	Tests verifying dynamic policy reload; integration test updating a policy and seeing effect	
Low	Documentation & matrices	Create kernel documentation describing modules, extensions, plugins, and lifecycle; document boundary mapping and plugin interfaces; produce traceability matrices	New docs exist; DMOS developers can understand kernel usage; diagrams updated	Manual review; docs included in repo	
Low	Strategic backlog	Continue with phases 5–6 of hardening plan: build configuration service, event registry, stable rule DSL, generic CLI, plugin marketplace	Implementation milestones added to roadmap; design documents created	N/A – long-term	

6. Conclusion

The kernel provides a robust foundation for building domain-agnostic products like DMOS, offering pluggable services for authorization, compliance, risk, approval, ledger, consent, and observability. However, product-specific implementations have crept into DMOS, duplicating kernel logic and preventing generic reuse. To align with the hardening plan, the team should prioritise consolidating policy resolvers, generalising risk/compliance engines, wiring observability ports, implementing encryption/hashing, and validating domain packs. These enhancements will ensure DMOS can leverage the kernel's full power and support multiple products with consistent governance and operational behaviour.

1 2 [KERNEL_MODULE_EXTENSION_PLUGIN_DECISION_GUIDE.md](#)

https://github.com/samuijwal/ghatana/blob/4431520ab5902d5b306088aec1cdcde8e4477e22/docs/kernel/KERNEL_MODULE_EXTENSION_PLUGIN_DECISION_GUIDE.md

3 4 5 6 7 [KERNEL_BOUNDARY_HARDENING_PLAN.md](#)

https://github.com/samuijwal/ghatana/blob/4431520ab5902d5b306088aec1cdcde8e4477e22/KERNEL_BOUNDARY_HARDENING_PLAN.md